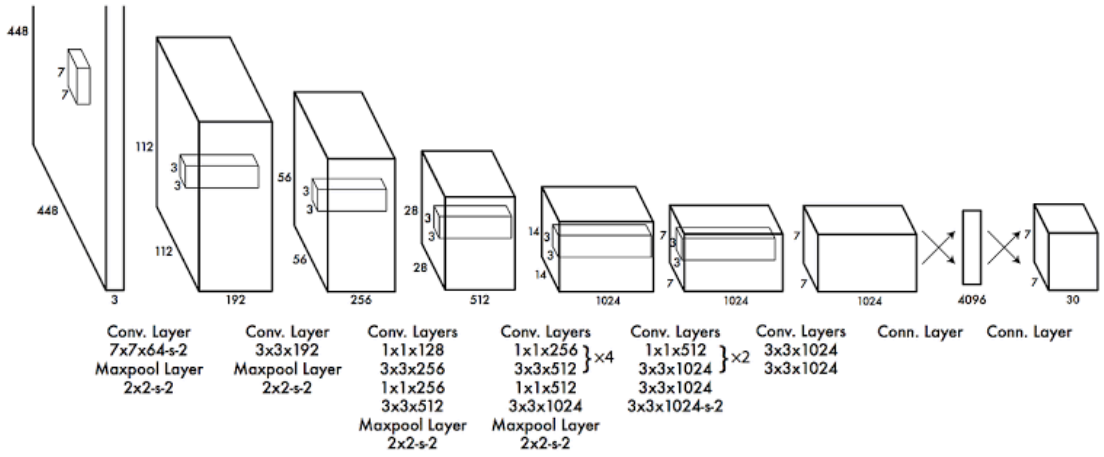


1. The Input Layer

The network requires a fixed input size. Any image fed into YOLO is first resized to **448x448 pixels** with 3 color channels (RGB). This resolution was specifically chosen to be double the standard ImageNet classification size (224x224) to give the network finer granular detail for drawing precise bounding boxes around small objects.

2. The Convolutional Backbone (Feature Extraction)

The core of the network consists of **24 Convolutional layers** separated by periodic Max Pooling layers. This section's job is to extract visual features—starting with simple edges and textures in the early layers, and building up to complex object shapes in the deeper layers.

- **Alternating Convolutions:** Instead of GoogLeNet's complex Inception modules, YOLO uses a highly efficient alternating structure. It chains together **1x1 convolutional layers** and **3x3 convolutional layers**.
- **The 1x1 Bottleneck:** The 1x1 layers act as mathematical bottlenecks. They compress the depth (the number of feature channels) of the data right before it enters the heavier 3x3 layers. This drastically reduces the computational load, which is exactly what allows YOLO to run at real-time speeds.
- **Spatial Downsampling:** Through a series of 2x2 Max Pooling layers, the spatial dimensions of the image are repeatedly halved. By the time the data exits the 24th convolutional layer, the 448x448 image has been compressed into a deep, low-resolution feature map measuring exactly **7x7x1024**.

4. The Output Reshape

Final Fully Connected layer outputs a flat vector of exactly **1,470 neurons**. Those final 1,470 neurons are mathematically reshaped back into the 3D tensor of shape **7x7x30**.

- **7x7** represents the spatial grid overlaid on the image.
- **30** represents the data for each grid cell: 2 bounding boxes (5 values each: x, y, width, height, confidence) plus 20 class probabilities (based on the 20 object categories in the PASCAL VOC dataset YOLO was trained on).

The Training Process

- **The Grid:** The network logically divides the input image into an S x S grid. The absolute governing rule of YOLO is: whichever grid cell contains the exact mathematical center of an object is strictly responsible for detecting it.
- **The Output Tensor:** The final layer of the network condenses the image into a 3D mathematical tensor of shape S x S x (B x 5 + C).
- **Tensor Breakdown:** For every single cell, the network predicts B bounding boxes. Each box contains 5 values: x and y (center coordinates bound between 0 and 1), w and h (width and height, which can exceed 1), and a **confidence score**. Finally, the cell predicts C conditional class probabilities.
- **Ground Truth Assignment (The Competition):** For every real object in a training image, YOLO identifies the grid cell containing its center. If that cell predicts multiple boxes, YOLO calculates which box has the highest Intersection over Union (IoU) with the true human-drawn box. Only that "winning" box is deemed responsible and receives weight updates for that object.

Training YOLO is a delicate balancing act. Because the network is predicting coordinates, object presence, and class labels all at once, it uses a unique, multi-part **Sum of Squared Errors** loss function to update its weights.

The Complete YOLOv1 Loss Equation

$$
\begin{aligned}
L_{\text{YOLOv1}} = & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbb{1}_{i}^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned}
$$

Inference and Cleanup

During real-world inference, this architecture allows YOLO to process the entire image in one forward pass, generating hundreds of dense bounding box predictions in milliseconds. Because the grid structure naturally causes multiple adjacent cells to mistakenly guess at the same object, the architecture relies on **Non-Maximum Suppression (NMS)** at the very end to filter out the overlapping, lower-confidence boxes and leave only the cleanest, most accurate prediction.